

BOOKPOOL DEVELOPER AND SYSTEM DESCRIPTION

By Jaiden Lo, Allan Zhang, Cooper Ross

Overview

BookPool is a full-stack MERN web application (MongoDB, Express, React, Node.js) that functions as a virtual library. Users can list books for borrowing, borrow and return books, join a waiting queue, and participate in per-book discussion threads. The application runs fully containerized using Docker Compose.

File Structure

The project is split into two top-level folders: client and server, with a docker-compose.yml and .env file at the root.

The client folder contains the React frontend built with Vite. All source files live under src/, which is divided into pages/ and components/. Pages are full route-level views such as Home, Threads, Profile, etc.. Components are reusable pieces used across pages, such as Nav, Footer, BookCard. Each page and component has a corresponding CSS file. A tests/ folder contains the Vitest unit test files. App.jsx handles all routing and defines the ProtectedRoute and AdminRoute guards.

The server folder contains the Express backend. Controllers handle the business logic and are split into bookController.js, threadController.js, and userController.js. Models define the MongoDB schemas for Book, Thread, and User. Routes map HTTP endpoints to controller functions. A middleware folder contains validate.js which handles server-side input validation before requests reach the controllers.

How the Application Works

Authentication & States

User state is stored in localStorage as a JSON object after login and registration. App.jsx reads this on load and passes it down as a user prop. Two route guards are used. ProtectedRoute (any logged in user) and AdminRoute (admin only). Passwords are hashed with bcrypt before being stored in MongoDB.

Book Borrowing and Queues

When a user borrows a book, the borrowed field is set to true and borrowedBy is set to the user's ID. If a book is already borrowed, other users can join a queue array on the book document. When the book is returned, the server automatically shifts the first user from the queue and sets a reservedFor object with a 24 hour expiry to claim. Only the reserved user can borrow the book during that timeframe. The home page automatically polls every 5 seconds to reflect changes made by other users without a page refresh making it asynchronous.

Images

User profile images and book cover images are stored as base64-encoded strings directly into MongoDB. On registration, the image is converted to base64 on the frontend and sent as part of the JSON body. Profile images can be updated from the profile page at any time. The maximum image size for each field is 2mb.

Discussion Threads

Threads are stored in a separate Thread collection, each linked to a bookId. The thread page polls every 5 seconds for new posts. Every new thread shows the username, date/time posted, and profile picture. Users can delete their own posts and admins can delete any post. If a book is deleted, the threads on that are also deleted and no longer show up on your profile.

Sorting and Filtering

Books can be filtered by category using a dropdown that triggers instantly without clicking the search button. Books can also be sorted by newest, hottest (threads #) and availability. The polling interval pauses when a filter is active to avoid overwriting a user's filtered view.

Admin

Admin users have access to the admin dashboard with Recharts charts and can delete users, delete any thread posts, and edit any book. The admin role can be set directly from the database.

FEATURES IMPLEMENTED

- User registration and login with bcrypt password hashing
- Profile picture upload at registration and from the profile page
- Book CRUD with image upload and category filtering
- Instant category filter without needing to press Search
- Book search by title and description
- Sort books by newest, hottest (thread count), or available first
- Borrow and return system with a waiting queue and 24-hour reservation
- Per-book discussion threads with 5-second async polling
- Collapsible thread posts (hide/show without page reload)
- Breadcrumb navigation in threads linking back to the book modal
- Inline success and error alerts on AddBook and Threads pages
- Admin dashboard with usage charts (Recharts)
- Admin user search and deletion
- Admin ability to delete any thread post
- Admin ability to edit any book
- Responsive layout (3 columns on desktop, 2 on tablet, 1 on mobile)
- Frontend and server-side input validation
- Docker Compose setup for single-command deployment
- Test unit tests for validation logic, home helpers, and thread helpers

KNOWN LIMITATIONS

- Profile pictures and book images are stored as base64 strings in MongoDB rather than a dedicated file storage service, which limits image size and may affect performance at scale.
- The admin enable/disable user feature is not implemented — admins can delete users but cannot temporarily suspend them.
- The borrowing queue reservation expiry is not automatically enforced on the server — expired reservations are only cleared when the next borrow attempt is made.
- Thread posts do not support editing, only deletion.
- The polling interval pauses when filters are active, meaning live updates will not appear while a category filter is applied.

TESTING

Project uses Vitest for frontend unit testing located at `client/src/tests` and covers validation of login/registration, Homepage book ownership, queue membership etc, and threads test that test thread delete permissions, empty post detection and fetch call mocking for post and delete operations.

These tests must be ran locally outside of Docker using Vitest.

To run:

cd client

npm install

npx vitest run